# VIDEO TIMESTAMP EXTRACTION TOOL

## Operations & Setup Guide

Version 1.0

Produced: May 3, 2026

# 1. Overview

This tool scans a directory of video files, reads on-screen timestamp overlays using OCR (Optical Character Recognition), and produces a structured report of when each video was recorded. It is designed for CCTV, dashcam, or any video footage that embeds a visible date/time stamp in the frame.

## 1.1 What the Tool Does

- Scans a folder for video files automatically — no manual file listing required.
- Crops the top-left corner of each frame where the timestamp appears.
- Uses Tesseract OCR to read the timestamp text from each frame.
- Applies a majority-vote algorithm across all frames to determine the most reliable start time.
- Detects anomalies such as time jumps greater than 10 minutes or footage from different calendar days.
- Outputs a per-video summary log and a machine-readable results file.

## 1.2 Output Files

| File | Description |
|---|---|
| ocr_log_YYYYMMDD_HHMMSS.txt | Full diagnostic log with per-video summaries, detected timestamps by day, and anomaly warnings. A new file is created each run. |

## 1.3 Timestamp Format Supported

The tool is built to read timestamps in the following format as displayed on screen:

```
2026-01-22 20:42:21 Thur
```

> ℹ️ The day-of-week text (e.g. 'Thur') is ignored by the parser. Only the YYYY-MM-DD HH:MM:SS portion is extracted.

# 2. Dependencies & Installation

The tool requires Python 3.8 or higher and the following packages. Install all dependencies before running the script.

## 2.1 Python Packages

| Package | Purpose | Install Command |
|---|---|---|
| opencv-python | Video frame reading and image cropping | `pip install opencv-python` |
| Pillow | Image handling for OCR input | `pip install Pillow` |
| pytesseract | Python wrapper for Tesseract OCR engine | `pip install pytesseract` |
| argparse | Command-line argument parsing | Built into Python — no install needed |

Install all Python packages at once with:

```
pip install opencv-python Pillow pytesseract
```

## 2.2 Tesseract OCR Engine

Tesseract is a separate system-level application that pytesseract calls. It must be installed independently of Python.

### Windows

- Download the installer from: https://github.com/UB-Mannheim/tesseract/wiki
- Run the installer and note the installation path (default: C:\Program Files\Tesseract-OCR\).
- Add Tesseract to your system PATH, or set the path explicitly in the script by adding this line near the top:

```
import pytesseract
pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-OCR\
tesseract.exe'
```

> ⚠️ Without setting the path on Windows, pytesseract will fail with a 'tesseract is not installed or not in PATH' error.

### macOS

Install via Homebrew (recommended):

```
brew install tesseract
```

If Homebrew is not installed, get it first from https://brew.sh

### Linux (Ubuntu / Debian)

```
sudo apt update
sudo apt install tesseract-ocr
```

### Linux (Fedora / RHEL / CentOS)

```
sudo dnf install tesseract
```

## 2.3 Verify Tesseract Installation

Run the following in your terminal to confirm Tesseract is installed and accessible:

```
tesseract --version
```

You should see output similar to: tesseract 5.x.x

## 2.4 Full Dependency Summary Table

| Dependency | Windows | macOS | Linux |
|---|---|---|---|
| Python 3.8+ | `python.org installer` | `brew install python` | `sudo apt install python3` |
| `pip` | Included with Python | Included with Python | `sudo apt install python3-pip` |
| opencv-python | `pip install opencv-python` | `pip install opencv-python` | `pip install opencv-python` |
| Pillow | `pip install Pillow` | `pip install Pillow` | `pip install Pillow` |
| pytesseract | `pip install pytesseract` | `pip install pytesseract` | `pip install pytesseract` |
| Tesseract OCR | UB-Mannheim installer | `brew install tesseract` | `sudo apt install tesseract-ocr` |

# 3. Running the Tool

## 3.1 Basic Usage

Navigate to the folder containing the script and run:

```
python ts_extract.py -d /path/to/your/videos
```

| Argument | Required | Description |
|---|---|---|
| -d / --dir | Yes | Path to the folder containing your video files. |

## 3.2 Platform Examples

### Windows

```
python ts_extract.py -d "C:\Users\YourName\Videos\CCTV"
```

### macOS / Linux

```
python3 ts_extract.py -d /home/yourname/videos/cctv
```

## 3.3 Supported Video Formats

| Extension | Format |
|---|---|
| .mp4 | MPEG-4 — most common |
| .avi | Audio Video Interleave |
| .mov | Apple QuickTime |
| .mkv | Matroska Video |
| .wmv | Windows Media Video |

> ℹ️ Additional formats can be added by editing the SUPPORTED_EXTENSIONS tuple near the top of the script.

## 3.4 What to Expect During a Run

The tool will print a live summary to the console as each video is processed. A new timestamped log file is created in the working directory at the start of each run. Processing time depends on video length and frame count — a 17-second 30fps clip has approximately 510 frames.

# 4. Understanding the Output

## 4.1 Log File Summary Block

Each video produces a block in the log file like the following:

```
============================================================
VIDEO: footage_01.mp4
  Duration       : 17.0s at 30.0 FPS
  Total frames   : 510
  OCR successes  : 498
  OCR failures   : 12 frames
  Confidence     : 96.7%
  Mismatch frames: 17
  Origin stamp   : 2026-01-22 20:42:04

  DETECTED TIMESTAMPS BY DAY:
    📅 2026-01-22 (14 unique second(s) detected)
       First : 20:42:04
       Last  : 20:42:17
       Range : 0:00:13 duration
       20:42:04, 20:42:05, 20:42:06, 20:42:07, ...

  ✅  No anomalies — video appears to be a single continuous session.
============================================================
```

## 4.2 Anomaly Warning Example

If spliced or multi-session footage is detected, the log will include:

```
  ⚠️  ANOMALIES DETECTED — possible spliced/multi-session video!
    Frame    423: timestamp 2026-01-23 08:15:01 is on a DIFFERENT DAY
  ⚠️  MULTIPLE DATES FOUND: ['2026-01-22', '2026-01-23']
```

# 5. Configuration

The following constants near the top of the script can be adjusted without changing any logic:

| Constant | Default | Effect |
|---|---|---|
| JUMP_THRESHOLD_MINUTES | 10 | Minutes of drift from baseline before a frame is flagged as an anomaly. |
| SUPPORTED_EXTENSIONS | mp4, avi, mov, mkv, wmv | File types the tool will process in the target directory. |
| Crop region (height) | 12% of frame height | Controls how tall the cropped OCR region is from the top of the frame. |
| Crop region (width) | 45% of frame width | Controls how wide the cropped OCR region is from the left of the frame. |

## 5.1 Adjusting the Crop Region

If your timestamp is not being detected, the crop region may need to be adjusted. Find this section in GetTimestamp():

```
roi = frame[0:int(height * 0.12), 0:int(width * 0.45)]
```

The first pair (0 to 0.12) controls vertical coverage from the top. The second pair (0 to 0.45) controls horizontal coverage from the left. Increase these values if the timestamp is being cut off.

> ℹ️ Temporarily add 'print(text)' after the pytesseract call to see exactly what OCR is reading on each frame — this is the fastest way to diagnose crop issues.

# 6. Troubleshooting

| Symptom | Likely Cause | Fix |
|---|---|---|
| FAILED for all videos | Tesseract not found or not in PATH | Verify with 'tesseract --version'. On Windows, set the path explicitly in the script. |
| FAILED for all videos | Crop region misses the timestamp | Adjust the crop percentages in GetTimestamp(). Print OCR text to diagnose. |
| Low confidence (<70%) | Timestamp partially obscured or blurry | Check video quality. Increase crop area slightly. |
| [ANOMALY DETECTED] on normal footage | JUMP_THRESHOLD_MINUTES too low | Increase JUMP_THRESHOLD_MINUTES at the top of the script. |

| | | |
|---|---|---|
| No videos found in directory | File extensions not in SUPPORTED_EXTENSIONS | Add the extension to the SUPPORTED_EXTENSIONS tuple. |
| OCR failures very high (>30%) | Video has frames without a visible timestamp | Normal for intro/outro frames. Check confidence % — above 70% is reliable. |

## 7. Expected File Structure

Before running the tool, your working directory should look like this:

```
project/
├── timestamp_extractor.py     ← The script
└── videos/                    ← Your video folder (passed with -d)
    ├── footage_01.mp4
    ├── footage_02.mp4
    └── footage_03.avi
```

After running, the directory will contain:

```
project/
├── timestamp_extractor.py
├── stamp_file.txt                          ← Results summary
├── ocr_log_20260122_204221.txt              ← Detailed diagnostic log
└── videos/
    ├── footage_01.mp4
    ├── footage_02.mp4
    └── footage_03.avi
```

> ℹ️ stamp_file.txt is opened in append mode — results from multiple runs accumulate in the same file. The log file is uniquely named per run so previous logs are never overwritten.